Patchwork

A road-defect register from ordinary dashcam video: damage measured in **m²** with **OpenCV 5**, tracked pass after pass, turned into repair work by an **AI agent** that cannot act without human approval.

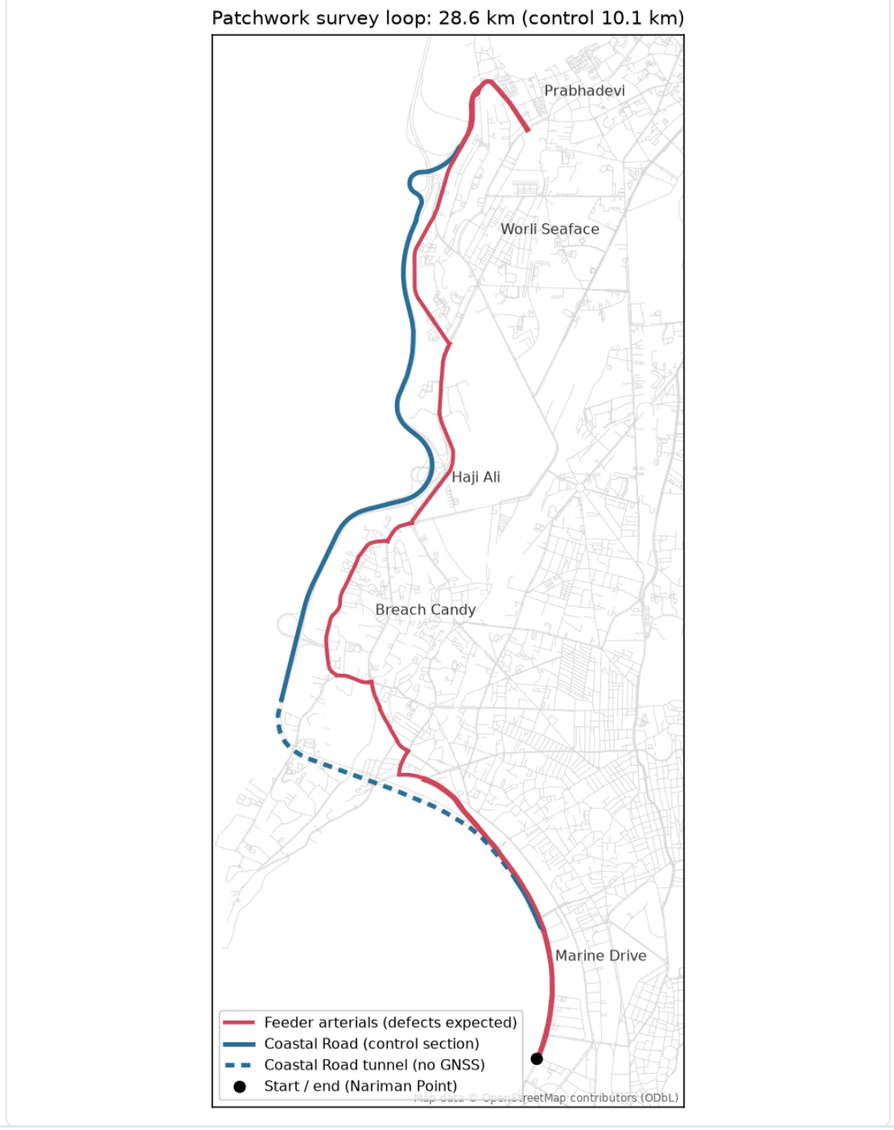
Mumbai · AWS ap-south-1 · OpenCV AI Competition 2026, Agentic Vision · Midpoint checkpoint · 14 October 2026

What a ward engineer needs, and what we built

Detecting a pothole is solved. Knowing **how big**, **where exactly**, and **whether it was really repaired** is not.

- **Measure**: dashcam frame → calibrated top-down view at 1 cm/pixel → damaged area in m² ± uncertainty
- **Locate**: on a named road, by chainage, also in the 3.1 km tunnel
- **Track**: the same defect across passes → new / unchanged / degraded / repaired
- **Act**: an Amazon Nova agent groups defects into work packages, estimates hot-mix, ranks by nearby schools and hospitals, and **requests human approval**

28.6 km real Mumbai loop: Marine Drive, Coastal Road, Worli Seaface, Haji Ali (OpenStreetMap geometry, 84 schools / hospitals / bus stops).



Exact ground truth: a simulated dashcam over the real road network

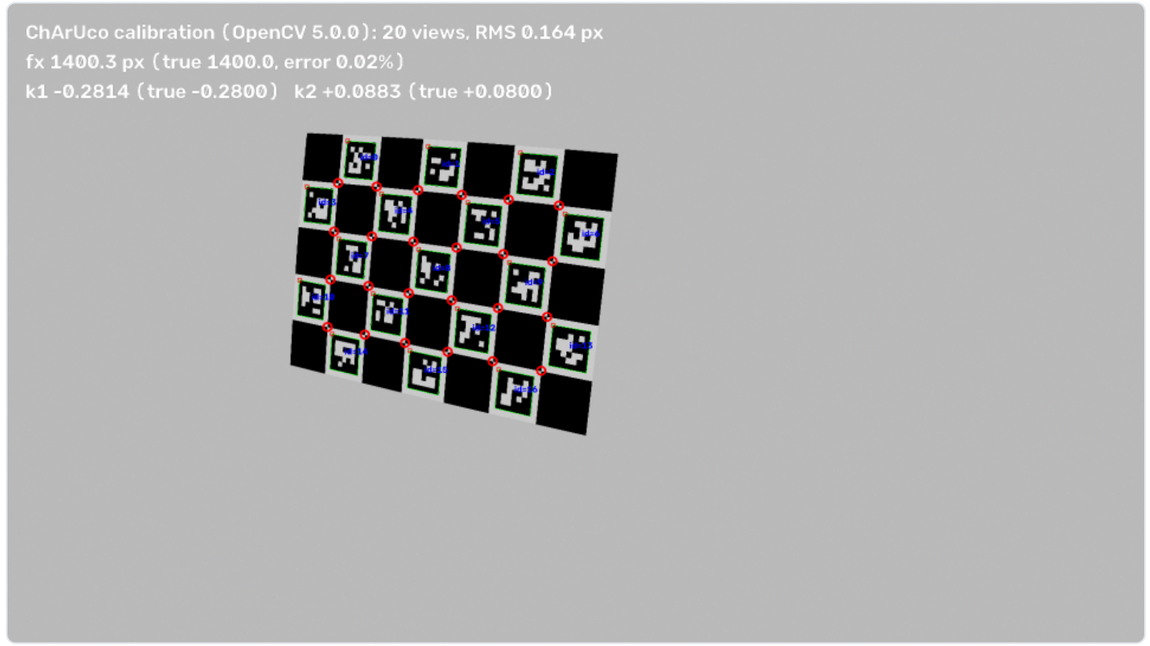
Every pothole's true area, position and repair status is known exactly, so every number below is graded against truth, not against a tape measure.



- 2 worlds × 3 passes × 28.6 km rendered on AWS Batch
- Seed 07 for training, **seed 11 held out** for every result
- Traps on purpose: shadows, wet patches, manholes, seams, old repairs
- Real photos for the model: Pothole Mix (4,340 masked images)
- The viewer states clearly that the defects are simulated

OpenCV 5 geometry: from pixels to square metres

ChArUco calibration (RMS 0.164 px) → per-frame road pitch from lane lines → bird's-eye view at 1 cm/px → area is a pixel count.





2.1% area error with perfect outlines (target ≤ 5%): the geometry is not the bottleneck. Frame triage keeps 284 frames/km and loses no defects; faces and plates are blurred before storage; median position error 2.6 m (raw GNSS 6.2 m).

The model: SegFormer-B0 running in OpenCV 5's new DNN engine

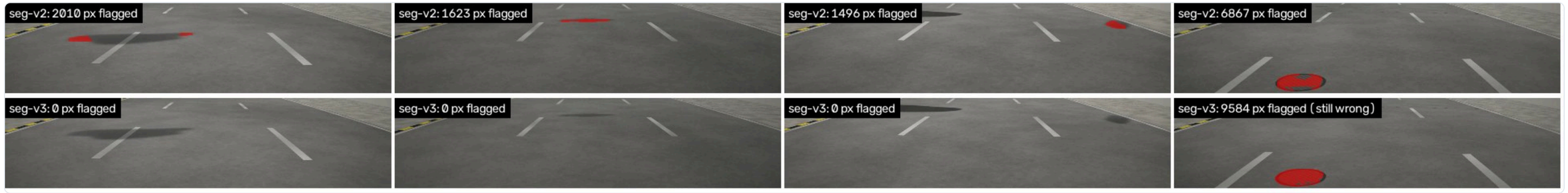
Trained on CPU only (AWS Graviton Spot). Runs in `ENGINE_NEW`, identical to onnxruntime (max difference 2.0e-5, target 1e-3). The classic 4.x-style engine cannot load it at all.



Top: real photos (held-out test split). Bottom: synthetic dashcam strips (held-out road stretch). Green = truth, red = model. Worst cases shown on purpose. IoU 0.50 real / 0.57 synthetic.

Fine-tune with look-alikes: teaching the model what is *not* damage

1,788 frames of shadows, wet patches, manholes, seams and old repairs labelled “not damage”; seg-v2 → seg-v3 (2.7 h CPU, survived 3 Spot interruptions by resuming from S3 checkpoints).

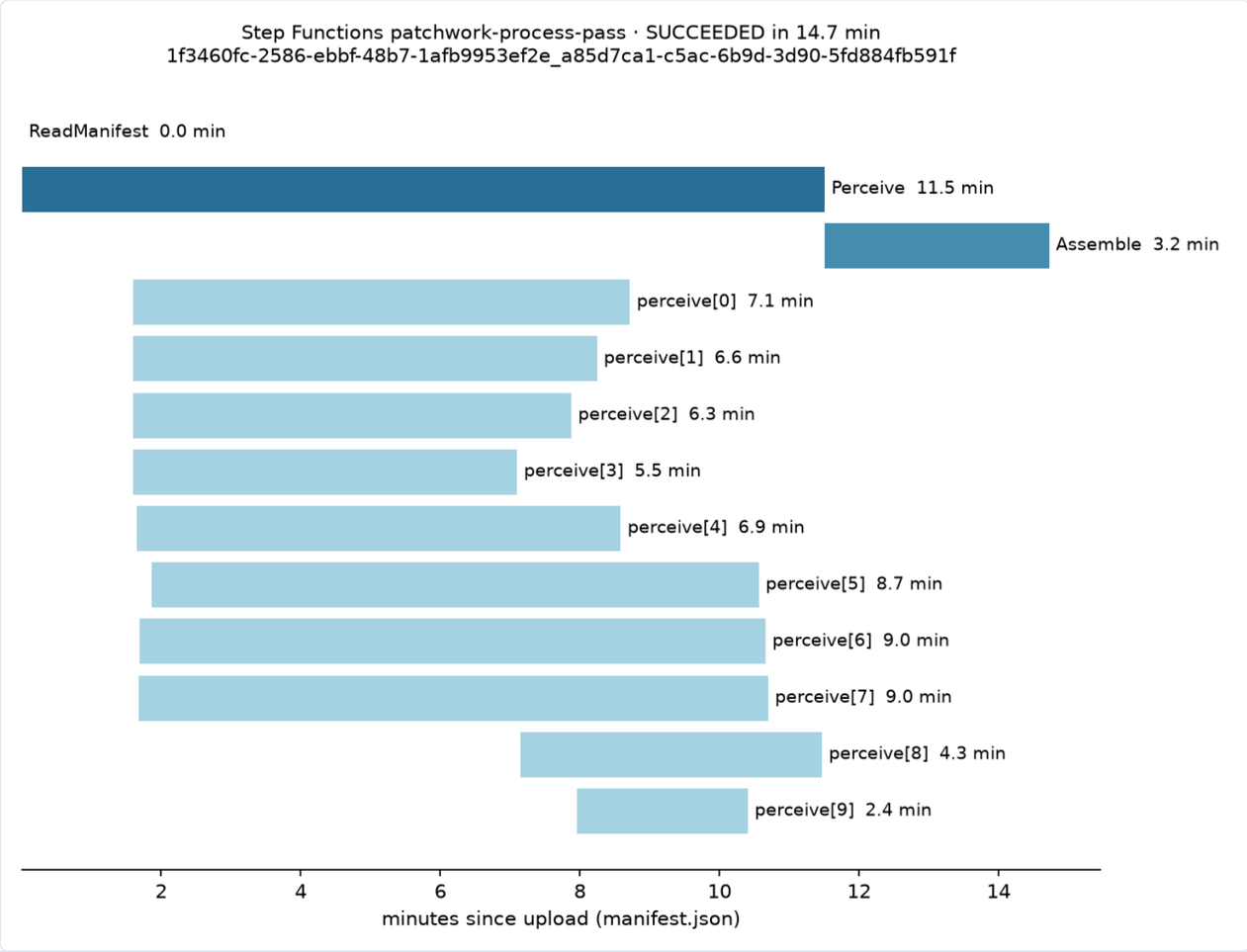


Held-out set	False-alarm images, seg-v2	seg-v3
Look-alikes only, no damage (480)	86	**41**
Real photos, test (318 without damage)	73	**54**
Synthetic Mumbai, val (719 without damage)	64	**23**

The cloud pipeline: uploading a survey is the trigger

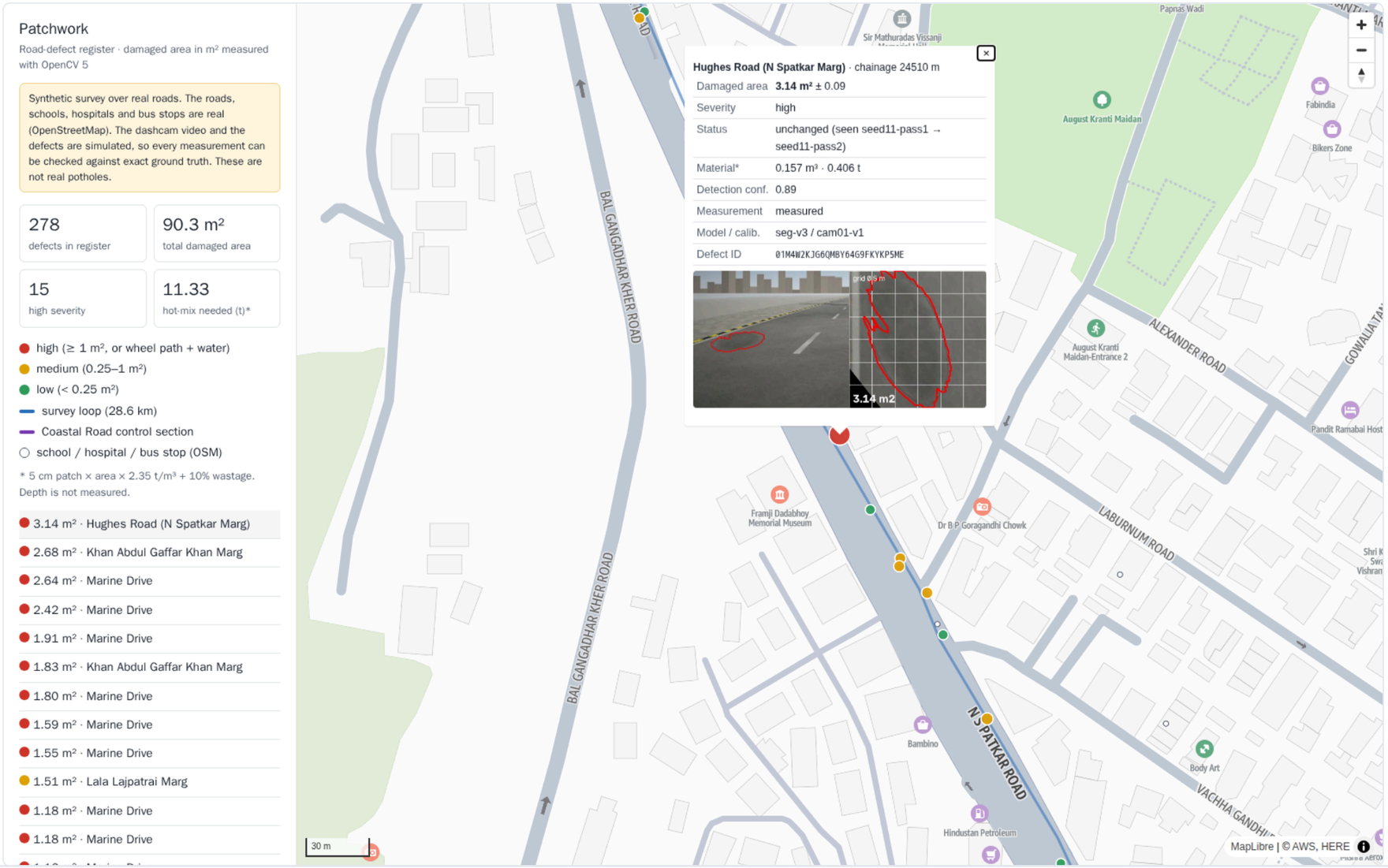
Held-out survey: 2 passes, 56.8 km. One 28.4 km pass in 15–17 min wall-clock; perception compute $0.0021 per km (target < $0.02).

S3 upload → EventBridge → Step Functions → AWS Batch, Graviton Spot, 10 clips in parallel (OpenCV 5) → Aurora PostgreSQL + PostGIS register → CloudFront map → Nova agent + approval gate



Live map: every defect in m², with its evidence

https://d3mt6jrs1ynczo.cloudfront.net · click a dot: area ± uncertainty, severity, status across passes, hot-mix estimate, and the dashcam view next to the measured top-down view.



Results on the held-out survey (56.8 km, exact ground truth)

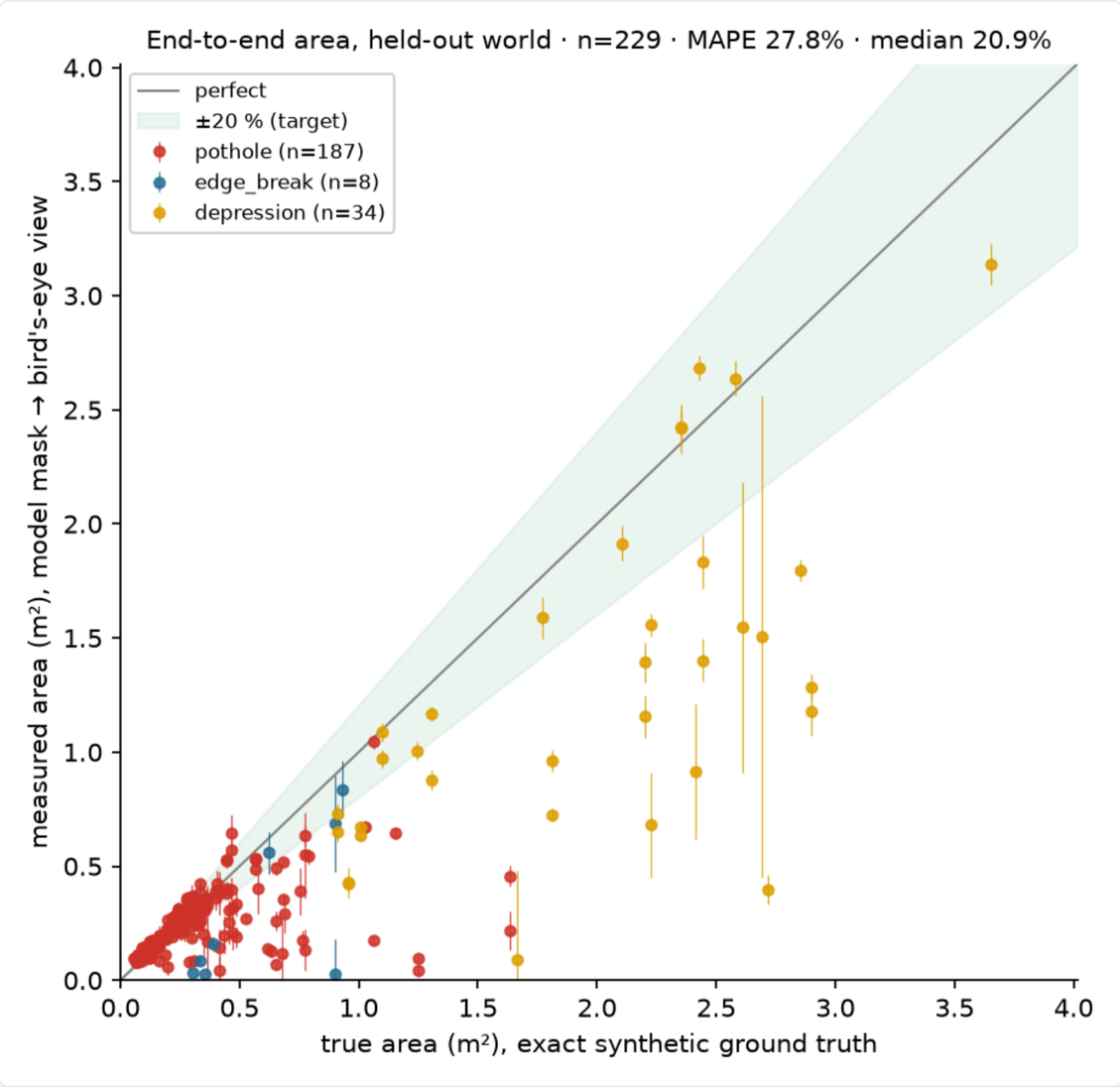
Same survey, same pipeline, before and after the look-alike fine-tune.

Metric	seg-v2	seg-v3	Target	
Recall (visible defects found)	0.95	**0.94**	≥ 0.85	**met**
False alarms per km, clean control section	3.87	**0.30**	≤ 0.5	**met**
False alarms per km, whole survey	10.05	**3.1**	–	
Area error (MAPE), end to end	33.7%	**27.8%**	≤ 20%	**not yet**
Area error, geometry only (perfect outlines)	2.1% (geometry, same for both)		≤ 5%	**met**
Re-identification across passes (F1)	0.57	**0.73**	≥ 0.90 (final)	**second half**
Median position error	2.6 m (localisation, same for both)		≤ 8 m	**met**
OpenCV 5 ENGINE_NEW vs onnxruntime	2.4e-5	**2.0e-5**	≤ 1e-3	**met**
Perception cost per km	$0.0019	**$0.0021**	< $0.02	**met**

Generated by `make eval-midpoint` from `eval/results/*.json` (04-RESULTS §1, §3).

Measured vs true area, every matched defect

Small and medium potholes sit near the diagonal (median error 18%). Large shallow depressions are still under-measured: the model marks only part of their soft edge.



Proof that vision drives the agent: run A vs run B

Same pothole (Marine Drive, chainage 3193 m), same prompt, same model (Amazon Nova Pro, temperature 0). Only the measured area differs.

	Run A (as measured by the pipeline)	Run B (area swapped)
Measured area	0.35 m² (true 0.35 m²)	3.10 m²
Priority (1 = urgent)	2	**1**
Exposure score	5.86	21.70
Hot-mix needed	0.046 t	**0.401 t**
Tool calls	query_defects → cluster_into_packages → estimate_materials → draft_work_order → request_human_approval	

Priority and quantities are computed by tools from the register, so the change is caused by the measurement, not by model randomness. Unknown defect IDs are rejected by the tool and by a database trigger.

Agent, run A:

"…medium severity, an area of 0.3531 m², and is located near two hospitals… The estimated materials required are 0.0177 m³ of hot-mix asphalt."

Agent, run B:

"…It has a priority of 1 due to its high severity, an area of 3.1 m², and its proximity to two hospitals. The estimated materials for this package are 0.155 m³ of hot-mix asphalt…"

Full traces: eval/results/agent/ablation-A-400AET.json, ablation-B-400AET.json

The human gate: the agent can only ask

A Step Functions workflow holds the approval token in the database; the agent never sees it.

Agent drafts work order → request_human_approval → Step Functions waits (task token, 7 days) → Engineer: approve / reject → filed: MOCK ticket · or · rejected: override recorded

- Road-wide request “Plan repairs for Khan Abdul Gaffar Khan Marg” → **24 work packages** in 78 s (84 k Nova Pro tokens ≈ $0.10)
- **Approved** (priority 1, 3.40 m²) → state `filed`, ticket `MOCK-aea1bb9b⋯`: nothing is sent to a real authority
- **Rejected** (0.02 m², “re-inspect first”) → state `rejected`, recorded as an override
- No answer → the request expires after 7 days

Honest note: the agent's final text summary listed one package twice. The packages, numbers and approvals in the database were correct, because code computes them, not the model's prose.

What the credits bought: first half

Every AWS service, step by step, is in 08-SERVICE-MAPPINGS. Billed from AWS Cost Explorer.

$0.00

real money paid; everything on promotional credits, guarded by a $0.01 budget alarm

- $9.23 billed so far (≈ $10.6 once Spot billing catches up)
- Perception: **$0.0021 per km**
- Model training on CPU: $1.7 (seg-v2) + $1.2 (seg-v3)
- 13 AWS services, 42 resources, all Terraform

Billed by AWS (2026-10-12 → 13)	Usage
EC2 compute: dev workstation $4.21 · Batch Spot $2.20	$6.43
S3 (requests, cross-region transfer)	$0.86
Aurora Serverless v2 register	$0.57
Cost Explorer API (this report)	$0.42
EC2 other (disks), VPC (public IPs)	$0.62
Bedrock: Amazon Nova Pro (agent)	$0.33
Secrets, ECR, Location, EventBridge, Step Functions	$0.01
Total, all paid by credits	**$9.23**

Second half: what the remaining credits pay for (≈ $370, range $260–470)

The first half proved the pipeline at the cheapest settings. The second half buys accuracy and the full agent.

- **GPU training**: g6.xlarge (NVIDIA L4) via AWS Batch, SegFormer-B2, ≈ 110 GPU-hours → $80
- **40-scenario agent evaluation** on Amazon Nova, run live ≈ 50 times → $74; LLM judge + VLM attributes
- Aurora for 8-pass verification and the console → $40
- Development workstation → $92
- AgentCore, Lambda, API Gateway, Cognito, CloudWatch, S3, pipeline evaluation runs

- Fix manhole false alarms, merge split blobs
- Area error → 20%: soft-boundary labels, larger model, mask fusion
- Re-ID → 0.90: ORB + homography matcher over 8 passes
- Full 10-tool agent on AgentCore, web console
- Caps on every line; no padded spend